



Compte rendu de TP 5

Java EE



Module : *Application Web Avancées avec Java*

Filière : *Transformation Digitale & Intelligence Artificielle 2.*

Année : *2025/2026.*

Réalisé par :

- Es-saouiqui Amine.

Supervisée par :

- Dr. Cherradi Mohamed



Table de Matière :

I.	Introduction :	5
II.	Front Controller Design-Pattern :	5
I.	Architecture technique :	6
II.	Réalisation technique :	7
1)-	Organisation du projet :	7
a)	Couche Backend (src/main/java).....	8
b)	Couche Configuration (src/main/resources)	9
c)	Couche Frontend (src/main/webapp)	9
2)-	Configuration du fichier pom.xml :	9
3)-	Configuration du fichier hibernate.cfg.xml :	10
3)-	Définition des entités :	12
a)	– User.hbm.xml :	12
b)-	Produit.hbm.xml :	13
4)-	Couche DAO :	14
a)	– BaseDAO.java :	14
b)-	Interface ProduitDAO.java :	15
c)-	Interface UserDAO.java :	16
d)-	ProduitDAOImpl.java :	16
e)	– UserDAOImpl.java :	18
5)	- Couche Service :	18
6)-	Configuration de SessionFactory :	19
7)-	Couche controllers :	20
a)	– Class Command.java :	21
b)	– Interface Controller.java :	22
c)-	Classe ProduitController.java :	22
d)	– Classe UserController.java :	26
8)-	Dispatcher :	28
9)-	Front Controller :	31
10)-	Configuration du web.xml :	32
a)	- Configuration des paramètres globaux	32



11)- Les filtres :	33
a)- AuthFilter.java :	33
b) – RoleFilter.java :	35
c)- HibernateSessionFilter.java :	36
12)- Définition des vues :	39
a) – index.jsp :	39
b) home.jsp :	40
c)- error.jsp :	42
III. Conclusion :	43

Table de Figure :

Figure 1 : flux des requêtes par le FrontController	5
Figure 2 : Diagram de Class du FrontController Design Pattern	6

Table de Scripts :

Script 1 : Contenu de fichier pom.xml	10
Script 2 : Configuration du fichier hibernate.xml	11
Script 3 : Mapping d'entité User.java	12
Script 4 : Mapping d'entité Produit.java	13
Script 5 : La classe BaseDAO.java	15
Script 6 : interface ProduitDAO.java	16
Script 7 : Interface UserDAO.java	16
Script 8 : La classe ProduitDAOImpl.java	17
Script 9 : La classe UserDAOImpl.java	18
Script 10 : La classe HibernateUtils.java	20
Script 11 : La classe Command.java	21
Script 12 : Interface Controller.java	22
Script 13 : Classe ProduitController.java	25
Script 14 : classe UserController.java	28
Script 15 : La classe Dispatcher.java	30
Script 16 : La classe FrontControllerServlet.java	32
Script 17 : Contenu du fichier web.xml	33
Script 18 : Classe AuthFilter.java	34
Script 19 : Classe RoleFilter.java	36
Script 20 : Classe HibernateSessionFilter.java	38
Script 21 : la définition de la page index.jsp	40
Script 22 : home.jsp	42



Script 23 : error.jsp	42
-----------------------------	----



I. Introduction :

Ce projet porte sur le développement d'une application web de gestion de stock réalisé dans le TP précédent supportant l'ensemble des opérations **CRUD** sur les produits. L'architecture a été conçue selon le modèle **MVC-2**, articulé autour du design pattern **FrontController**. Cette approche centralise le traitement des requêtes pour garantir une séparation rigoureuse des préoccupations et permet une gestion fine de la sécurité et des accès en fonction du profil de l'utilisateur.

II. Front Controller Design-Pattern :

Le **Front Controller** est un design pattern qui centralise la gestion de toutes les requêtes d'une application web via un point d'entrée unique.

Plutôt que d'avoir une multitude de fichiers accessibles directement, toutes les requêtes HTTP sont interceptées par un composant central (**souvent un Servlet Java**)

Ses rôles principaux :

- **Centralisation** : *Il reçoit toutes les demandes des utilisateurs.*
- **Aiguillage (Dispatching)** : *Il analyse la requête et délègue le travail au contrôleur spécifique capable de traiter la demande.*

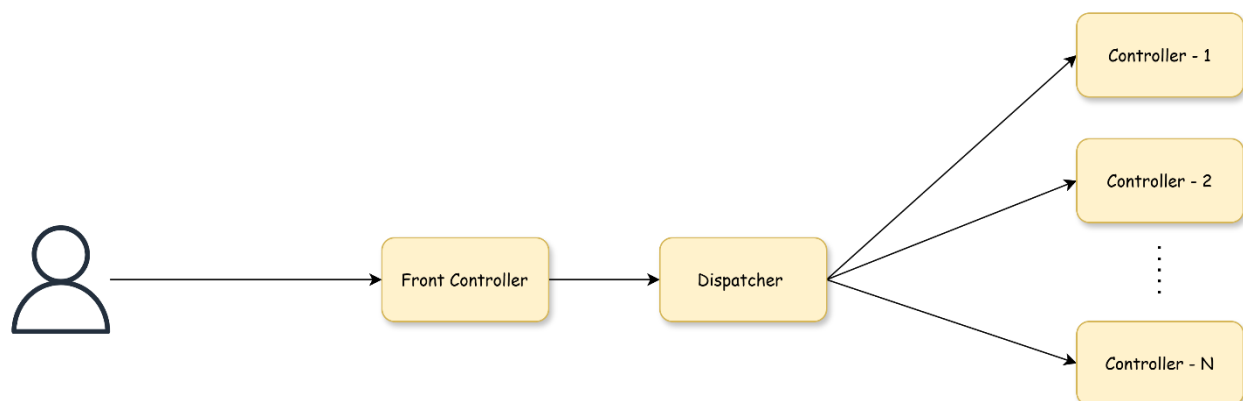


Figure 1 : flux des requêtes par le FrontController

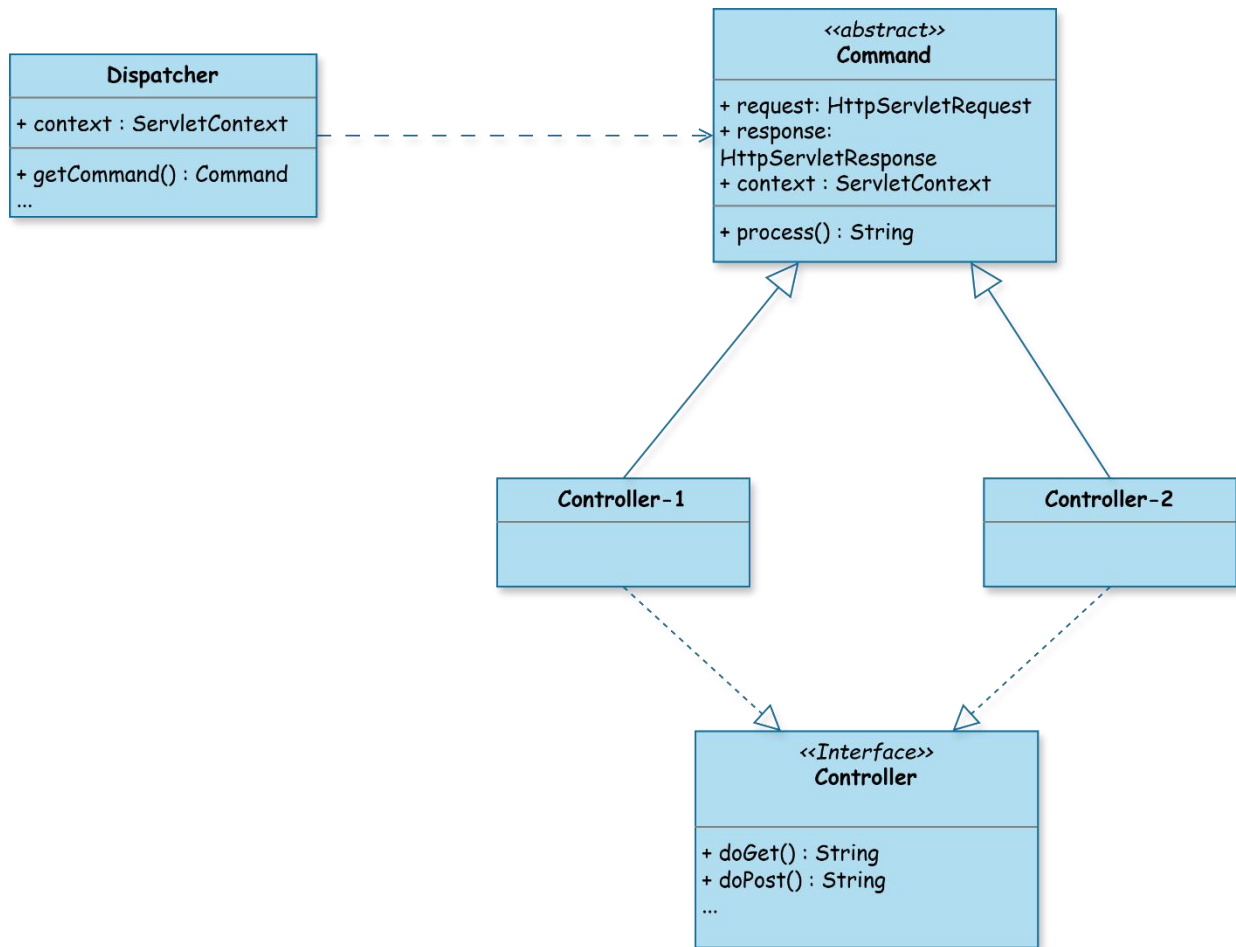


Figure 2 : Diagram de Class du FrontController Design Pattern

I. Architecture technique :

L'application est une application JEE pure utilisant les mêmes fonctionnalités que celles développées lors du TP précédent.

- **Contrôleur & Vue (JSP)** : Les pages JSP reçoivent et traitent les requêtes. L'utilisation de la JSTL permet de gérer la logique d'affichage sans recourir à du code Java embarqué dans le HTML.
- **Modèle** : Les données sont encapsulées dans des POJO, assurant une séparation claire entre les différentes couches de l'application.



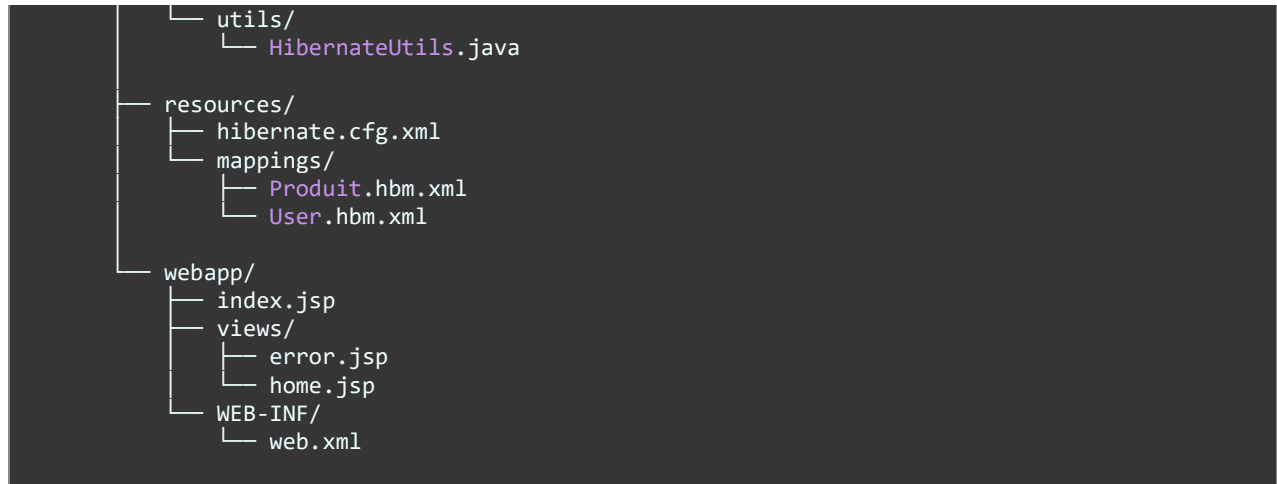
- **Persistence** : Contrairement au TP précédent, la couche de persistance a été améliorée par l'intégration de Hibernate, permettant une gestion plus efficace et abstraite de l'accès à la base de données via le mapping objet-relationnel (ORM).

II. Réalisation technique :

1)- Organisation du projet :

Le projet respecte l'arborescence standard d'une application web de type **Maven**, permettant une séparation nette entre le code source, les ressources et l'interface utilisateur.

```
tp4_mvc2:/
├── pom.xml
├── src/
│   └── main/
│       ├── java/
│       │   ├── controllers/
│       │   │   ├── Command.java
│       │   │   ├── Controller.java
│       │   │   ├── ProduitController.java
│       │   │   └── UserController.java
│       │   ├── dao/
│       │   │   ├── BaseDAO.java
│       │   │   ├── ProduitDAO.java
│       │   │   ├── ProduitDAOImpl.java
│       │   │   ├── UserDAO.java
│       │   │   └── UserDAOImpl.java
│       │   ├── dispatchers/
│       │   │   └── Dispatcher.java
│       │   ├── entities/
│       │   │   ├── Produit.java
│       │   │   ├── Roles.java
│       │   │   └── User.java
│       │   ├── filters/
│       │   │   ├── AuthFilter.java
│       │   │   ├── HibernateSessionFilter.java
│       │   │   └── RoleFilter.java
│       │   ├── services/
│       │   │   ├── ProduitService.java
│       │   │   ├── ProduitServiceImpl.java
│       │   │   ├── UserService.java
│       │   │   └── UserServiceImpl.java
│       │   └── servlets/
│       │       └── FrontControllerServlet.java
```



a) Couche Backend (src/main/Java)

Ce répertoire constitue le cœur de la logique applicative, organisé en plusieurs packages respectant une architecture en couches :

- **entities** : Regroupe les objets de données . Les classes comme **Produit.java** et **User.java** définissent la structure des données et facilitent leur circulation entre les différentes couches de l'application.
- **dao** : Ce package isole la logique de persistance. Il repose sur des interfaces (**ProduitDAO**, **UserDAO**) et leurs implémentations (**ProduitDAOImpl**, **UserDAOImpl**), en s'appuyant sur **Hibernate** pour assurer le mapping objet-relationnel et l'accès à la base de données.
- **services** : Joue le rôle d'intermédiaire entre les contrôleurs et les DAO. Il contient la logique métier, notamment les règles de gestion et les validations nécessaires avant toute interaction avec la base de données.
- **controllers** : Regroupe les classes responsables du traitement des requêtes métier. Elles définissent les actions à exécuter en fonction des commandes reçues et participent à l'organisation du flux applicatif.
- **servlets** : Contient le contrôleur principal (**FrontControllerServlet**), implémentant le pattern **Front Controller**. Il centralise toutes les requêtes HTTP et les délègue aux contrôleurs appropriés.
- **dispatchers** : Assure la gestion du routage interne des requêtes. Le **Dispatcher** permet de rediriger dynamiquement vers les bonnes actions ou vues selon le contexte.



- **filters** : Package dédié à la gestion des aspects transversaux. **AuthFilter** vérifie l'authentification des utilisateurs, **RoleFilter** contrôle les autorisations, et **HibernateSessionFilter** gère le cycle de vie des sessions Hibernate.
- **utils** : Contient des classes utilitaires, notamment **HibernateUtils** qui permet de configurer et de gérer la **SessionFactory** pour l'accès à la base de données.

b) Couche Configuration (src/main/resources)

Ce répertoire regroupe les fichiers de configuration nécessaires au bon fonctionnement de l'application :

- **hibernate.cfg.xml** : Fichier principal de configuration de Hibernate, définissant les paramètres de connexion à la base de données ainsi que les propriétés ORM.
- **mappings** : Contient les fichiers de mapping (**Produit.hbm.xml**, **User.hbm.xml**) qui assurent la correspondance entre les classes Java et les tables de la base de données.

c) Couche Frontend (src/main/webapp)

Ce répertoire représente la partie visible de l'application, gérée par le serveur web :

- **views** : Contient les pages JSP (**comme home.jsp et error.jsp**), utilisées pour afficher les données dynamiques à l'utilisateur via JSTL.
- **WEB-INF** : Répertoire sécurisé contenant le fichier **web.xml**, qui configure les servlets, filters et le cycle de vie de l'application. Ce dossier n'est pas accessible directement via URL.
- **index.jsp** : Point d'entrée principal de l'application, servant de page d'accueil ou de redirection initiale.

2)- Configuration du fichier pom.xml :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>tp4_mvc2</artifactId>
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>Maven Webapp Archetype</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
```



```
</dependency>
<!-- Source: https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
  <scope>compile</scope>
</dependency>
<!-- Source: https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
  <scope>compile</scope>
</dependency>
<!-- Source: https://mvnrepository.com/artifact/javax.servlet/jstl -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
  <scope>compile</scope>
</dependency>
<!-- Source: https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>3.1.0</version>
  <scope>provided</scope>
</dependency>
</dependencies>
<build>
  <finalName>tp4_mvc2</finalName>
</build>
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
</project>
```

Script 1 : Contenu de fichier pom.xml

3)- Configuration du fichier hibernate.cfg.xml :

Avant de détailler les différents paramètres de configuration, il est important de comprendre le rôle central du fichier **hibernate.cfg.xml** dans l'architecture de l'application. En effet, ce fichier constitue le point de liaison entre l'application Java et la base de données, en assurant la configuration du framework **Hibernate**. Il permet de définir les propriétés essentielles liées à la connexion, au comportement de l'ORM ainsi qu'à la gestion des entités. Ainsi, il joue un rôle clé dans la mise en œuvre de la couche de persistance, en garantissant une interaction fluide, cohérente et transparente entre les objets Java et les tables relationnelles.



```
<?xml version='1.0' encoding='utf-8'?>
<hibernate-configuration>
  <!-- a SessionFactory instance listed as /jndi/name -->
  <session-factory>
    <!-- properties -->
    <!-- JDBC Database Connection Settings -->
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/tp4_db</property>
    <property name="hibernate.connection.username">amine</property>
    <property name="hibernate.connection.password">1234</property>
    <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
    <property name="show_sql">false</property>
    <!-- Enable Hibernate to automatically create/drop tables -->
    <property name="hibernate.hbm2ddl.auto">update</property>
    <!-- Additionnal : Relating a session context to one thread -->
    <property name="hibernate.current_session_context_class">thread</property>
    <!-- mapping files -->
    <mapping resource="mappings/Produit.hbm.xml"/>
    <mapping resource="mappings/User.hbm.xml"/>
  </session-factory>
</hibernate-configuration>
```

Script 2 : Configuration du fichier hibernate.xml

Voici un explication détaillé de cette configuration :

- **Configuration de la connexion JDBC** : Les propriétés `hibernate.connection.driver_class`, `hibernate.connection.url`, `hibernate.connection.username` et `hibernate.connection.password` définissent les informations nécessaires pour établir la connexion avec la base de données **MySQL**. Elles précisent respectivement le driver utilisé, l'URL de la base (`tp4_db`), ainsi que les identifiants d'accès.
- **Dialecte Hibernate** : La propriété `hibernate.dialect` spécifie le dialecte SQL utilisé, ici **MySQL8Dialect**. Cela permet à Hibernate de générer des requêtes SQL adaptées au système de gestion de base de données utilisé.
- **Affichage des requêtes SQL** : La propriété `show_sql` permet d'activer ou non l'affichage des requêtes SQL générées dans la console. Dans cette configuration, elle est désactivée (**false**) pour éviter la surcharge des logs.
- **Gestion du schéma de la base de données** : La propriété `hibernate.hbm2ddl.auto` est définie sur `update`. Cela permet à Hibernate de mettre à jour automatiquement le schéma de la base de données en fonction des entités, sans supprimer les données existantes.
- **Gestion du contexte de session** : La propriété `hibernate.current_session_context_class` est définie sur `thread`, ce qui signifie que chaque session Hibernate est liée à un thread d'exécution, facilitant ainsi la gestion des transactions dans une application web.



- **Fichiers de mapping** : Les balises `<mapping>` permettent de déclarer les fichiers de mapping XML (`Produit.hbm.xml` et `User.hbm.xml`). Ces fichiers assurent la correspondance entre les classes Java et les tables de la base de données.

3)- Définition des entités :

Pour la définition des entités, nous avons conservé les mêmes classes que celles utilisées dans le TP précédent, notamment `Produit` et `User`, afin de maintenir la cohérence du modèle de données. Toutefois, la différence majeure réside dans la manière de configurer leur persistance. En effet, au lieu d'utiliser des annotations directement dans les classes Java, le mapping objet-relationnel a été externalisé à l'aide de fichiers XML (`Produit.hbm.xml` et `User.hbm.xml`). Cette approche, supportée par Hibernate, permet de séparer clairement la logique métier de la configuration de persistance, offrant ainsi une meilleure modularité et une plus grande flexibilité dans la gestion des correspondances entre les entités Java et les tables de la base de données.

a) – `User.hbm.xml` :

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
  <class name="entities.User" table="USERS">
    <id name="id" type="long" column="id">
      <generator class="identity"/>
    </id>
    <property name="username" type="string" column="username"/>
    <property name="password" type="string" column="password"/>
    <property name="role" column="role">
      <type name="org.hibernate.type.EnumType">
        <param name="enumClass">entities.Roles</param>
      </type>
    </property>
  </class>
</hibernate-mapping>
```

Script 3 : Mapping d'entité User.java



Le fichier de mapping `User.hbm.xml` définit la correspondance entre la classe Java `User` et la table relationnelle `USERS` de la base de données. Ce fichier est utilisé par Hibernate afin d'assurer le mapping objet-relationnel sans recourir aux annotations dans le code source.

- **Déclaration de la classe et de la table :** La balise `<class>` associe la classe `entities.User` à la table `USERS`, permettant ainsi la persistance des objets `User` dans la base de données.
- **Clé primaire (id) :** L'attribut `id` est défini comme clé primaire via la balise `<id>`. Il est de type long et utilise un générateur `identity`, ce qui signifie que sa valeur est auto-incrémentée par la base de données.
- **Propriétés simples :** Les attributs `username` et `password` sont mappés directement aux colonnes correspondantes de la table, avec des types simples (**string**).
- **Gestion de l'énumération role :** L'attribut `role` est mappé à une colonne de type **ENUM** en utilisant `org.hibernate.type.EnumType`. Cette configuration permet de stocker les valeurs de l'énumération Java `entities.Roles` sous forme lisible en base de données, tout en conservant leur typage côté application.

b)- Produit.hbm.xml :

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
  <class name="entities.Produit" table="PRODUITS">
    <id name="id" type="long" column="id">
      <generator class="identity"/>
    </id>
    <property name="nom" type="string" column="nom"/>
    <property name="description" type="string" column="description"/>
    <property name="prix" type="double" column="prix"/>
  </class>
</hibernate-mapping>
```

Script 4 : Mapping d'entité Produit.java



Le fichier de mapping **Produit.hbm.xml** définit la correspondance entre la classe Java **Produit** et la table relationnelle **PRODUITS** dans la base de données. Ce mapping est utilisé par Hibernate afin d'assurer la persistance des objets de manière transparente, sans recourir aux annotations dans le code source.

- **Déclaration de la classe et de la table** : La balise **<class>** associe la classe **entities.Produit** à la table **PRODUITS**, ce qui permet de stocker les objets **Produit** directement dans la base de données.
- **Clé primaire (id)** : L'attribut **id** est défini comme identifiant unique de l'entité via la balise **<id>**. Il est de type **long** et utilise un générateur **identity**, permettant une génération automatique de la clé primaire par la base de données.
- **Propriétés de l'entité** : Les attributs **nom**, **description** et **prix** sont mappés respectivement aux colonnes de la table avec des types adaptés (**string** pour les textes et **double** pour les valeurs numériques).

4)- Couche DAO :

Dans ce projet, la couche DAO est structurée autour d'interfaces et de leurs implémentations, permettant de définir clairement les opérations CRUD (Create, Read, Update, Delete) tout en masquant les détails techniques liés à la base de données. L'accès aux données est géré via Hibernate, ce qui permet de manipuler les entités Java sans écrire directement de requêtes SQL.

a) – **BaseDAO.java** :

Elle constitue une classe abstraite de base pour la couche DAO. Elle joue un rôle central dans la factorisation et la gestion de l'accès à la base de données au sein de l'application.

Dans cette implémentation, la gestion de la persistance repose sur Hibernate, ce qui permet d'utiliser des sessions pour interagir avec la base de données de manière orientée objet.



```
package dao;

import org.hibernate.Session;
import org.hibernate.SessionFactory;

import utils.HibernateUtils;

public abstract class BaseDAO {
    protected Session session;
    protected static SessionFactory factory;
    static {
        factory = HibernateUtils.getSessionFactory();
    }
}
```

Script 5 : La classe BaseDAO.java

- **Session Hibernate** : L'attribut `session` représente une session **Hibernate**, utilisée pour effectuer les opérations **CRUD** (création, lecture, mise à jour et suppression) sur les entités.
- **SessionFactory** : L'attribut statique `factory` est une instance de **SessionFactory**, responsable de la création des sessions **Hibernate**. Elle est initialisée une seule fois dans un bloc statique afin d'optimiser les performances et éviter la création répétée de l'objet.
- **Initialisation centralisée** : La **SessionFactory** est obtenue via la classe utilitaire **HibernateUtils**, ce qui permet de centraliser la configuration **Hibernate** et de garantir une gestion cohérente des connexions à la base de données.

b)- Interface ProduitDAO.java :

L'interface **ProduitDAO** définit l'ensemble des opérations de persistance liées à l'entité **Produit**.

```
package dao;
import java.util.List;

import entities.Produit;

public interface ProduitDAO {
    public void addProduct(Produit p);
    public void deleteProduit(Long p);
    public Produit findProduitById(Long id);
    public void updateProduit(Produit p );
    public List<Produit> findAllProduits();
}
```



Script 6 : interface ProduitDAO.java

Cette interface s'appuie sur Hibernate via ses implémentations, afin de manipuler les entités de manière orientée objet tout en assurant leur stockage en base de données.

- **addProduct(Produit p)** : Permet d'ajouter un nouveau produit dans la base de données.
- **deleteProduit(Long p)** : Permet de supprimer un produit identifié par son identifiant.
- **findProduitById(Long id)** : Permet de rechercher et récupérer un produit à partir de son identifiant unique.
- **updateProduit(Produit p)** : Permet de mettre à jour les informations d'un produit existant.
- **findAllProduits()** : Permet de récupérer la liste de tous les produits présents dans la base de données.

c)- Interface UserDAO.java :

L'interface **UserDAO** définit les opérations de persistance liées à l'entité **User**.

```
package dao;

import entities.User;

public interface UserDAO {
    public User findByUsernameAndPassword(String username,String password);
}
```

Script 7 : Interface UserDAO.java

- **findByUsernameAndPassword(String username, String password)** : Cette méthode permet de rechercher un utilisateur dans la base de données en fonction de son nom d'utilisateur et de son mot de passe. Elle est principalement utilisée pour l'authentification des utilisateurs dans l'application.

d)- ProduitDAOImpl.java :

La classe **ProduitDAOImpl** représente l'implémentation concrète de l'interface **ProduitDAO**. Elle hérite de la classe **BaseDAO** afin de bénéficier de la gestion centralisée des sessions Hibernate et implémente les opérations de persistance liées à l'entité **Produit**.



```
package services;
import java.util.List;
import dao.ProduitDAO;
import dao.ProduitDAOImpl;
import entities.Produit;

public class ProduitServiceImpl implements ProduitService {
    private static ProduitDAO dao = new ProduitDAOImpl();
    @Override
    public void addProduct(Produit p) {
        dao.addProduct(p);
    }
    @Override
    public void deleteProduit(Long p) {
        dao.deleteProduit(p);
    }
    @Override
    public Produit getProduitById(Long id) {
        return dao.findProduitById(id);
    }
    @Override
    public void updateProduit(Produit p) {
        dao.updateProduit(p);
    }
    @Override
    public List<Produit> getAllProduits() {
        return dao.findAllProduits();
    }
}
```

Script 8 : La classe ProduitDAOImpl.java

- **addProduct(Produit p)** : Cette méthode permet d'ajouter un nouveau produit dans la base de données en utilisant **session.save()**.
- **deleteProduit(Long id)** : Cette méthode supprime un produit à partir de son identifiant. Elle utilise une requête **HQL (DELETE FROM Produit WHERE id= :id)** afin d'exécuter la suppression.
- **findAllProduits()** : Cette méthode récupère la liste de tous les produits enregistrés en base de données via une requête **HQL (from Produit)**.
- **findProduitById(Long id)** : Cette méthode permet de récupérer un produit précis à partir de son identifiant en utilisant **session.get()**.
- **updateProduit(Produit p)** : Cette méthode met à jour les informations d'un produit existant en utilisant **session.update()**.



e) – UserDAOImpl.java :

La classe `UserDAOImpl` représente l'implémentation concrète de l'interface `UserDAO`. Elle hérite de la classe `BaseDAO`, ce qui lui permet de bénéficier de la gestion centralisée des sessions Hibernate, et elle implémente les opérations de persistance liées à l'entité `User`.

```
package dao;

import entities.User;

public class UserDAOImpl extends BaseDAO implements UserDAO{

    @Override
    public User findByUsernameAndPassword(String username, String password) {
        session = factory.getCurrentSession();
        String query = "FROM User WHERE username= :u AND password= :p";
        User user = (User) session.createQuery(query)
            .setParameter("u", username)
            .setParameter("p", password)
            .uniqueResult();
        return user;
    }
}
```

Script 9 : La classe UserDAOImpl.java

- **findByUsernameAndPassword(String username, String password) :** Cette méthode permet de rechercher un utilisateur en fonction de son nom d'utilisateur et de son mot de passe. Elle utilise une requête **HQL (FROM User WHERE username = :u AND password = :p)** avec des paramètres nommés afin de sécuriser et structurer la requête. La méthode **uniqueResult()** est utilisée pour récupérer un seul utilisateur correspondant aux critères, ce qui est particulièrement adapté pour les opérations d'authentification.

5) - Couche Service :

La couche Service reprend les mêmes définitions et structures de classes que celles implémentées dans le TP précédent. Elle conserve donc la même organisation ainsi que les mêmes responsabilités, sans modification majeure au niveau de la logique métier.



6)- Configuration de SessionFactory :

La classe `HibernateUtils` a pour principal objectif de centraliser et simplifier la configuration de Hibernate au sein de l'application. Elle permet notamment d'éviter la lourdeur et la répétition du processus d'initialisation de la `SessionFactory`, qui est une opération coûteuse en ressources.

En effet, l'initialisation de Hibernate nécessite plusieurs étapes (*chargement de la configuration, création du `ServiceRegistry`, construction des métadonnées et génération de la `SessionFactory`*).

Cette classe encapsule toute cette logique dans un bloc statique exécuté une seule fois lors du chargement de la classe.

Ainsi, son utilité principale est de :

- Centraliser la configuration Hibernate
- Éviter la répétition de l'initialisation lourde de la `SessionFactory`
- Garantir une instance unique et réutilisable dans toute l'application
- Faciliter l'accès à la couche de persistance via une méthode simple

```
package utils;

import org.hibernate.SessionFactory;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;

public class HibernateUtils {

    private static SessionFactory factory;

    static {

        ServiceRegistry registry = null;
```



```
        try {
            registry = new
StandardServiceRegistryBuilder().configure().build();
            Metadata metadata = new
MetadataSources(registry).getMetadataBuilder().build();
            factory = metadata.getSessionFactoryBuilder().build();

        } catch (Exception e) {
            if (registry != null) {
                StandardServiceRegistryBuilder.destroy(registry);
            }
            throw new ExceptionInInitializerError(e);
        }
    }

    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```

Script 10 : La classe HibernateUtils.java

7)- Couche controllers :

Le package **controllers** implémente le pattern **Command**, où chaque classe représente une action métier spécifique encapsulée sous forme de commande. Contrairement à une architecture MVC classique basée uniquement sur des servlets, cette approche permet de structurer les traitements en unités indépendantes et réutilisables.

Dans cette application, les contrôleurs sont invoqués par un **Front Controller**, qui intercepte les requêtes HTTP et délègue leur traitement à la commande appropriée à l'aide du **Dispatcher**.

Chaque commande est donc responsable d'une fonctionnalité précise.



a) – Class Command.java :

La classe **Command** constitue la classe abstraite de base du package **controllers** et représente le cœur du pattern **Command** utilisé dans l'application. Elle définit une structure commune à toutes les commandes, permettant d'uniformiser le traitement des requêtes HTTP au sein du système.

Cette classe est directement intégrée dans l'architecture basée sur le **Front Controller**, qui se charge de recevoir toutes les requêtes et de les déléguer aux commandes appropriées. Ainsi, chaque commande hérite de cette classe et implémente sa propre logique de traitement.

```
package controllers;

import javax.servlet.ServletContext;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public abstract class Command {
    protected HttpServletRequest req;
    protected HttpServletResponse rep;
    protected ServletContext context;

    public void init(HttpServletRequest req, HttpServletResponse
rep, ServletContext context){
        this.req = req;
        this.rep = rep;
        this.context = context;
    }

    public abstract String process();
}
```

Script 11 : La classe Command.java

Elle encapsule les objets essentiels au traitement d'une requête web, à savoir :

- **HttpServletRequest** : représente la requête HTTP envoyée par le client et permet d'accéder aux paramètres et données transmises.
- **HttpServletResponse** : représente la réponse HTTP qui sera retournée au client après traitement.
- **ServletContext** : fournit un contexte global partagé par l'ensemble de l'application.



La méthode `init()` permet d'initialiser ces objets avant l'exécution de la commande. Elle est appelée par le **Front Controller** afin de préparer le contexte d'exécution.

Enfin, la méthode abstraite `process()` définit le point d'entrée du traitement de chaque commande. Elle est redéfinie dans les classes concrètes afin d'exécuter la logique métier correspondante et de retourner la vue ou la redirection appropriée.

b) – Interface Controller.java :

L'interface **Controller** définit un contrat commun pour les différentes commandes du package controllers. Elle s'inscrit dans l'implémentation du pattern **Command** et permet de standardiser la gestion des requêtes HTTP au sein de l'application.

Cette interface joue un rôle important dans l'architecture basée sur le **Front Controller**, en imposant une séparation claire entre le traitement des requêtes de type **GET** et **POST**.

```
package controllers;

public interface Controller {
    public String doGet();
    public String doPost();
}
```

Script 12 : Interface Controller.java

Elle contient deux méthodes principales :

- **doGet()** : Cette méthode est destinée à gérer les requêtes HTTP de type GET. Elle est généralement utilisée pour les opérations de consultation ou d'affichage des données.
- **doPost()** : Cette méthode est destinée à traiter les requêtes HTTP de type POST. Elle est principalement utilisée pour les opérations de création ou de mise à jour des données.

c)- Classe ProduitController.java :

La classe **ProduitController** constitue une implémentation concrète du pattern **Command** dans la couche controllers. Elle permet de gérer l'ensemble des requêtes HTTP liées à l'entité Produit et s'intègre dans l'architecture globale basée sur le **Front Controller**, où elle représente une unité de traitement spécialisée.



Cette classe étend la classe abstraite **Command** et implémente l'interface **Controller**, ce qui lui permet de structurer le traitement des requêtes en distinguant les opérations de type GET et POST.

Elle s'appuie sur la couche service via **ProduitService**, ce qui permet de déléguer la logique métier et de maintenir une séparation claire des responsabilités.

```
package controllers;

import java.util.List;

import entities.Produit;
import services.ProduitService;
import services.ProduitServiceImpl;

public class ProduitController extends Command implements Controller{

    private ProduitService service;

    public ProduitController(){
        service = new ProduitServiceImpl();
    }

    @Override
    public String process() {
        String method = req.getMethod();
        return switch (method){
            case "GET" -> this.doGet();
            case "POST" -> this.doPost();
            default -> "redirect:/Produit";
        };
    }

    @Override
    public String doGet() {
        String action = req.getParameter("action");
        if (action == null) action = "";
        String view = switch(action){
            case "getProduit" -> getProduit();
            case "deleteProduit" -> deleteProduit();
            case "editProduit" -> editProduit();
            default -> getAll();
        };
        return view;
    }
}
```



```
@Override
public String doPost() {
    String action = req.getParameter("action");
    if (action == null) return "redirect:Produit";
    String view = switch(action){
        case "addProduit" -> addProduit();
        case "updateProduit"-> updateProduct();
        default -> "redirect:/Produit";
    };
    return view;
}

private String getProduit(){
    String idParam = req.getParameter("id");
    if (idParam!=null && !idParam.isEmpty()){
        try{

            Long id = Long.parseLong(idParam.trim());
            Produit p = service.getProduitById(id);
            if(p!=null){
                List<Produit> products = List.of(p);
                req.setAttribute("Produits", products);
                return "home";
            }
        }catch(NumberFormatException nfe){}
    }
    return getAll();
}

private String deleteProuit(){
    String idParam = req.getParameter("id");
    if (idParam!=null && !idParam.isEmpty()){
        Long id = Long.parseLong(idParam.trim());
        service.deleteProduit(id);
    }
    List<Produit> products = service.getAllProduits();
    req.setAttribute("Produits", products);
    return "redirect:/Produit";
}
```




```
private String addProduit(){
    String nom = req.getParameter("nom").trim();
    String description = req.getParameter("description").trim();
    Double prix = Double.parseDouble(req.getParameter("prix"));
    service.addProduct(new Produit(nom, description,prix));
    List<Produit> products = service.getAllProduits();
    req.setAttribute("Produits",products);
    return "home";
}

private String updateProduct(){
    Long id = Long.parseLong(req.getParameter("id"));
    String nom = req.getParameter("nom");
    String description = req.getParameter("description");
    Double prix = Double.parseDouble(req.getParameter("prix"));
    Produit p = new Produit(nom,description,prix);
    p.setId(id);
    service.updateProduit(p);
    List<Produit> products = service.getAllProduits();
    req.setAttribute("Produits", products);
    return "home";
}

private String editProduit(){
    String prodId = req.getParameter("id").trim();
    Long id = Long.parseLong(prodId);
    Produit p = service.getProduitById(id);
    req.setAttribute("produitEdit", p);
    req.setAttribute("Produits", service.getAllProduits());
    return "home";
}

private String getAll() {
    req.setAttribute("Produits", service.getAllProduits());
    return "home";
}
}
```

Script 13 :Classe ProduitConroller.java



Pour le fonctionnement général :

- **process()** : Cette méthode constitue le point d'entrée de la commande. Elle détermine le type de requête HTTP reçue et redirige le traitement vers `doGet()` ou `doPost()`.
- **doGet()** : Cette méthode gère les requêtes HTTP de type GET et permet principalement les opérations de consultation et d'affichage des produits (liste, détail, suppression, édition).
- **doPost()** : Cette méthode gère les requêtes HTTP de type POST et est utilisée pour les opérations de modification des données telles que l'ajout et la mise à jour des produits.

Pour les autres fonctionnalités :

- **getProduit()** : récupération d'un produit par son identifiant
- **getAll()** : affichage de la liste de tous les produits
- **addProduit()** : ajout d'un nouveau produit
- **updateProduct()** : mise à jour des informations d'un produit existant
- **deleteProduit()** : suppression d'un produit
- **editProduit()** : préparation des données pour modification.

d) – Classe **UserController.java** :

La classe **UserController** constitue une implémentation du pattern **Command** dans la couche controllers. Elle est dédiée à la gestion des requêtes liées à l'authentification et à la gestion de la session utilisateur. Elle s'intègre dans l'architecture basée sur le **Front Controller**, en représentant une commande spécialisée pour les opérations utilisateur.

Cette classe étend **Command** et implémente l'interface **Controller**, ce qui lui permet de traiter les requêtes HTTP en séparant les méthodes GET et POST. Elle s'appuie sur la couche service via **UserService**, afin de déléguer la logique métier liée aux utilisateurs.



```
package controllers;

import javax.servlet.http.HttpSession;

import entities.User;
import services.UserService;
import services.UserServiceImpl;

public class UserController extends Command implements Controller {

    UserService service;

    public UserController(){
        this.service = new UserServiceImpl();
    }

    @Override
    public String process() {
        String method = req.getMethod();
        return switch (method){
            case "GET" -> this.doGet();
            case "POST" -> this.doPost();
            default -> "redirect:/index.jsp";
        };
    }

    @Override
    public String doGet() {
        String action = req.getParameter("action");
        String view = switch(action){
            case "logout"-> logout();
            default -> "home";
        };
        return view;
    }

    @Override
    public String doPost() {
        String action = req.getParameter("action");
        String view = switch(action){
            case "login" -> login();
            default -> "redirect:/index.jsp";
        };
        return view;
    }
}
```



```
private String login(){
    String username = req.getParameter("username");
    String password = req.getParameter("password");
    User user = service.login(username, password);
    if(user != null){
        HttpSession session = req.getSession();
        session.setAttribute("user", user);
        return "redirect:/Produit";
    }
    return "redirect:/index.jsp";
}

private String logout(){
    HttpSession session = req.getSession(false);
    session.invalidate();
    return "redirect:/index.jsp";
}
}
```

Script 14 : classe UserController.java

- **process()** : Détermine le type de requête HTTP et redirige vers `doGet()` ou `doPost()`.
- **doGet()** : Gère les opérations liées à la session, notamment la déconnexion de l'utilisateur.
- **doPost()** : Traite les opérations d'authentification, principalement la connexion.

8)- Dispatcher :

La classe **Dispatcher** joue un rôle central dans l'architecture de l'application basée sur le pattern **Front Controller**. Elle est responsable de l'aiguillage des requêtes HTTP vers les commandes appropriées du package **controllers**.

Son objectif principal est de déterminer dynamiquement la commande à exécuter à partir de l'URL de la requête, puis de gérer l'exécution de celle-ci ainsi que la redirection vers la vue correspondante.



Cette classe utilise la réflexion Java pour instancier dynamiquement les classes de type **Command**, en fonction du nom extrait de la requête HTTP. Elle repose également sur les paramètres de configuration définis dans le **ServletContext**, notamment le package des commandes et le chemin des vues.

```
package dispatchers;

import java.io.IOException;
import java.lang.reflect.InvocationTargetException;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import controllers.Command;

public class Dispatcher {

    private ServletContext context;

    public Dispatcher(ServletContext context ){
        this.context = context;
    }

    private Command getCommand(HttpServletRequest req) {
        String module = context.getInitParameter("commandPackage");
        String name = getCommandName(req);
        Command controller = null;
        try {
            Class <?> cl = Class.forName(module + "." + name + "Controller");
            controller = (Command) cl.asSubclass(Command.class) //
asSubclass() : cast the objet to subclass of the superclass type if it exists
                .getConstructor()
                .newInstance();
        }catch(ClassNotFoundException cfe){
            System.out.println("Class"+name+"Not Found");
        }catch (ClassCastException cce){
            System.out.println("Class"+name+"is Not a subclass");
        }catch( InstantiationException | IllegalAccessException |
                InvocationTargetException | NoSuchMethodException e){
            System.out.println("Cannot instantiate "+name);
        }
        return controller;
    }
}
```



```
private String getCommandName(HttpServletRequest req){
    String uri = req.getRequestURI();
    String name = uri.substring(uri.lastIndexOf("/") + 1 );
    return name;
}

public void dispatch(HttpServletRequest req, HttpServletResponse resp)
throws IOException, ServletException{
    Command controller = getCommand(req);
    if(controller == null){
        HandleError(req, resp, "Page "+getCommandName(req)+" Not Found");
        return;
    }
    controller.init(req, resp, context);
    String result = controller.process();
    if(result.startsWith("redirect:")){
        String view = result.split("redirect:")[1];
        resp.sendRedirect(req.getContextPath() + view);
    }else {
        String view = resloveView(result);
        RequestDispatcher dispatcher = req.getRequestDispatcher(view);
        dispatcher.forward(req, resp);
    }
}

private String resloveView(String view){
    String views = context.getInitParameter("viewsPath");
    return views + "/" + view + ".jsp" ;
}

private void HandleError(HttpServletRequest req, HttpServletResponse
resp, String message) throws ServletException, IOException{
    req.setAttribute("message", message);
    RequestDispatcher rd =
req.getRequestDispatcher(resloveView("error"));
    rd.forward(req, resp);
}
}
```

Script 15 : La classe Dispatcher.java



- **getCommand(HttpServletRequest req)** : Cette méthode permet de charger dynamiquement la classe de commande correspondante à la requête en utilisant la réflexion. Elle construit le nom complet de la classe, l'instancie et la retourne sous forme de *Command*.
- **getCommandName(HttpServletRequest req)** : Extrait le nom de la commande à partir de l'URL de la requête HTTP.
- **dispatch(HttpServletRequest req, HttpServletResponse resp)** : Méthode principale du dispatcher. Elle récupère la commande appropriée, l'initialise, puis exécute sa méthode *process()*. Selon le résultat retourné, elle effectue soit une redirection, soit un forwarding vers la vue correspondante.
- **resloveView(String view)** : Construit le chemin complet de la vue JSP à partir du nom logique de la vue et du chemin défini dans la configuration.
- **HandleError(...)** : Gère les erreurs lorsque la commande demandée n'existe pas en redirigeant vers une page d'erreur.

9)- Front Controller :

La classe **FrontControllerServlet** représente le point d'entrée principal de l'application web et implémente le pattern **Front Controller**. Elle hérite de la classe **HttpServlet** et intercepte toutes les requêtes HTTP grâce à son mapping racine **@ WebServlet("/")**.

Son rôle principal est de centraliser le traitement des requêtes et de les déléguer au **Dispatcher**, qui se charge ensuite d'orienter l'exécution vers les commandes appropriées.

```
package servlets;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import dispatchers.Dispatcher;
```



```
@WebServlet("/")
public class FrontControllerServlet extends HttpServlet{

    public static Dispatcher dispatcher;

    @Override
    public void init() throws ServletException {
        dispatcher = new Dispatcher(getServletContext());
        super.init();
    }

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
        dispatcher.dispatch(req, resp);
    }
}
```

Script 16 : La classe FrontControllerServlet.java

- **init()** : Cette méthode est exécutée lors du démarrage de l'application. Elle initialise une instance unique du **Dispatcher** en lui fournissant le **ServletContext**, ce qui permet de partager les paramètres de configuration de l'application.
- **service(HttpServletRequest req, HttpServletResponse resp)** : Cette méthode surcharge le comportement standard de la servlet. Elle intercepte toutes les requêtes HTTP (GET, POST, etc.) et les transmet directement au **Dispatcher** pour traitement.

10)- Configuration du web.xml :

Le fichier **web.xml** est utilisé pour définir les paramètres nécessaires au fonctionnement du **Front Controller** et du mécanisme de dispatching basé sur les commandes.

a) - Configuration des paramètres globaux

- **commandPackage** : Ce paramètre définit le package contenant les classes de type *Command* (controllers). Il est utilisé par le **Dispatcher** pour instancier dynamiquement les contrôleurs en fonction des requêtes HTTP.
- **viewsPath** : Ce paramètre indique le répertoire racine des vues JSP (views). Il permet au **Dispatcher** de construire dynamiquement le chemin complet des pages à afficher.



```
<!DOCTYPE web-app PUBLIC
"-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
"http://java.sun.com/dtd/web-app_2_3.dtd" >

<web-app>
  <display-name>Archetype Created Web Application</display-name>
  <context-param>
    <param-name>commandPackage</param-name>
    <param-value>controllers</param-value>
  </context-param>
  <context-param>
    <param-name>viewsPath</param-name>
    <param-value>views</param-value>
  </context-param>
  <error-page>
    <error-code>404</error-code>
    <location>/views/error.jsp</location>
  </error-page>
</web-app>
```

Script 17 : Contenu du fichier web.xml

11)- Les filtres :

a)- AuthFilter.java :

La classe **AuthFilter** fait partie de la couche **filters** et joue un rôle essentiel dans la sécurisation de l'application. Elle implémente l'interface **Filter** et permet d'intercepter les requêtes HTTP afin de vérifier l'authentification des utilisateurs avant d'accéder aux ressources protégées.

Ce filtre est appliqué sur l'URL **/Produit** ainsi sur la page d'accueil **home/views.jsp**, ce qui signifie que toutes les requêtes liées à la gestion des produits passent obligatoirement par ce mécanisme de contrôle.

```
package filters;

import java.io.IOException;

import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
```



```
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebFilter(urlPatterns = {
    "/Produit",
    "/views/home.jsp"
})
public class AuthFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;
        HttpSession session = req.getSession(false);
        boolean isLoggedIn = (session != null) && session.getAttribute("user")
!= null;

        if (isLoggedIn) {
            chain.doFilter(request, response);
        } else {
            resp.sendRedirect(req.getContextPath() + "/index.jsp");
        }
    }

    @Override
    public void destroy() {
    }
}
```

Script 18 : Classe AuthFilter.java



b) – RoleFilter.java :

La classe **RoleFilter** fait partie de la couche **filters** et permet d'implémenter un contrôle d'autorisation basé sur les rôles des utilisateurs. Elle joue un rôle complémentaire au filtre d'authentification en vérifiant non seulement si l'utilisateur est connecté, mais aussi s'il possède les privilèges nécessaires pour effectuer certaines actions.

Ce filtre est appliqué sur l'URL **/Produit**, ce qui signifie que toutes les opérations liées à la gestion des produits passent par un contrôle d'accès basé sur le rôle de l'utilisateur.

```
package filters;

import java.io.IOException;

import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

import entities.Roles;
import entities.User;

@WebFilter("/Produit")
public class RoleFilter implements Filter{

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;
        HttpSession session = req.getSession(false);
```



```
User user = (User) session.getAttribute("user");
boolean isAdmin = user.getRole() == Roles.ADMIN;
String action = req.getParameter("action");
if(action!=null){
    switch (action){
        case
"editPrdduit","addProduit","updateProduit","deleteProduit" -> {
            if(isAdmin) {
                chain.doFilter(request, response);
            }
            else {
                resp.sendRedirect(req.getContextPath()+"/Produit?acti
on=getProduit");
            }
        }
        default -> chain.doFilter(request,response);
    };
}
}else chain.doFilter(request, response);
}

@Override
public void destroy() {
}
}
```

Script 19 : Classe RoleFilter.java

c)- HibernateSessionFilter.java :

La classe **HibernateSessionFilter** fait partie de la couche **filters** et assure la gestion automatique des sessions et des transactions Hibernate pour les requêtes liées aux utilisateurs et aux produits. Elle joue un rôle essentiel dans la centralisation de la persistance en garantissant qu'une transaction est correctement ouverte et validée pour chaque requête traitée par l'application.

Ce filtre est appliqué sur les URLs **/User** et **/Produit**, ce qui signifie que toutes les opérations liées à ces ressources sont exécutées dans un contexte transactionnel unique et cohérent.



```
package filters;

import java.io.IOException;

import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;

import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import utils.HibernateUtils;

@WebFilter({
    "/User",
    "/Produit"
})
public class HibernateSessionFilter implements Filter{

    private SessionFactory factory;
    private Transaction tx;

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
        factory = HibernateUtils.getSessionFactory();
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException {
        try {
            tx = factory.getCurrentSession().beginTransaction();
            chain.doFilter(request, response);
            tx.commit();
        } catch (Exception e) {
            e.printStackTrace();
            throw new ServletException(e);
        }
    }
}
```



```
@Override
public void destroy() {
    if(factory != null) factory.close();
}
}
```

Script 20 : Classe HibernateSessionFilter.java

L'utilisation de ce filtre s'inscrit dans le **pattern session-per-request**, largement recommandé dans les applications web utilisant Hibernate. Ce modèle repose sur l'idée qu'une requête HTTP correspond à une **unit of work**, concept défini comme un ensemble d'opérations cohérentes sur la base de données, dans lequel les modifications sont coordonnées et validées de manière atomique.

Ainsi, chaque requête utilisateur est traitée dans une seule unité de travail :

- une Session Hibernate est ouverte au début de la requête,
- une transaction est démarrée automatiquement,
- toutes les opérations DAO s'exécutent dans ce même contexte,
- puis la transaction est validée (commit) à la fin du traitement.

Cette approche évite l'utilisation de l'antipattern **session-per-operation**, où une session serait ouverte et fermée pour chaque accès à la base de données. Une telle pratique entraîne une mauvaise performance, une complexité accrue et une incohérence dans la gestion des transactions.

En garantissant qu'une transaction encadre l'ensemble des opérations liées à une requête, ce filtre assure :

- la cohérence des données,
- une meilleure gestion de la concurrence,
- et une architecture plus maintenable et extensible.



12)- Définition des vues :

Pour les vues. Nous avons gardé les templates utilisés dans le TP précédent, que nous avons changé les URL avec :

- **/Produit** : pour gérer les produits.
- **/User** : pour gérer les utilisateurs.

Les actions de gestion de chaque entité est définie via l'attribut **action**.

a) – Index.jsp :

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ page isELIgnored="false" %>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.m
in.css" rel="stylesheet" integrity="sha384-
sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bun
dle.min.js" integrity="sha384-
FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
crossorigin="anonymous"></script>
    <title>Document</title>
</head>
<body>
    <section class="container mt-5 col-12">
        <div class="card col-10 ms-2">
            <h2 class="card-header text-center">Connectez-vous !!</h2>
            <div class="card-body">
                <form action=<c:url value="/User?action=login" />
method="post">
                    <label class="form-label">Nom d'utilisateur :
</label>
                    <input class="form-control" type="text"
name="username" required/>
                    <label class="form-label">Mot de passe : </label>
```



```

        <input class="form-control" type="password"
name="password" required/>
        <input type="submit" value="Connecter" class="btn
btn-primary mt-3">
    </form>
</div>
<div class="card-footer"></div>
</div>
</section>
</body>
</html>

```

Script 21 : la définition de la page index.jsp

b) home.jsp :

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ page isELIgnored="false" %>
<html>
<head>
    <title>Gestion des Produits : </title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
crossorigin="anonymous"></script>
</head>
<body>
<h2 class="display-6 p-5 bg-primary text-center col-6 text-light" style="margin : 10px 25%;
border-radius: 40px;">Gestion des Produits :</h2>
<section class="col-10 d-flex p-3 justify-content-evenly" style="margin: 5px 10%;">
<c:if test="${user.role == 'ADMIN'}" var="isAdmin">
    <div class="card mt-2 text-bg-success col-7">
        <h3 class="card-header">Creer ou modifier un produit : </h3>
        <div class="card-body">
            <form action=
            <c:url value="/Produit">
                <c:param name="action" value="${produitEdit != null ? 'updateProduit' :
'addProduit'}"/>
            </c:url>
                method="post">
                <input type="hidden" name="id" value="${produitEdit.id}" />
                <label class="form-label">Nom : </label>
                <input type="text" name="nom" value="${produitEdit.nom}" required class="form-
control"/>
                <label class="form-label">Description : </label>
                <input type="text" name="description" value="${produitEdit.description}"
required class="form-control"/>
                <label class="form-label">Prix:</label>
                <input type="text" name="prix" value="${produitEdit.prix}"
required class="form-control"/>

```




```

        <input type="submit" value="${produitEdit != null ? 'Modifier' : 'Ajouter'}"
class="btn btn-warning col-12 mt-2" />
    </form>
</div>
</div>
</c:if>

<div>
<div class="card text-bg-primary text-center mt-1" style="height: 200px;">
    <h2 class="card-header text-center">Chercher un Produit</h2>
    <div class="card-body d-flex align-items-center">
        <form action=<c:url value="/Produit"/> method="get">
            <div class="input-group">
                <input type="hidden" name="action" value="getProduit"/>
                <input type="text" name="id" class="form-control" placeholder="Entrer ID
du produit..." />

                <button class="btn btn-danger" type="submit">
                    Rechercher
                </button>
            </div>
        </form>
    </div>
</div>
<div class="mt-5 col-12 p-3">
    <button type="button" class="btn btn-danger col-12"><a href=<c:url
value='/User?action=logout'/>" class="text-decoration-none text-light">Logout </a></button>
</div>
</div>
</section>
<section>
    <h2>Liste des Produits : </h2>
    <table class="table table-hover table-dark">
        <tr>
            <th scope="col">ID</th>
            <th scope="col">Nom</th>
            <th scope="col">Description</th>
            <th scope="col">Prix</th>
            <c:if test="${isAdmin}" >
                <th scope="col">Actions</th>
            </c:if>
        </tr>
        <c:forEach var="p" items="${Produits}">
            <tr>
                <td>${p.id}</td>
                <td>${p.nom}</td>
                <td>${p.description}</td>
                <td>${p.prix}</td>
                <c:if test="${isAdmin}">
                    <td>
                        <a href=
                        <c:url value="/Produit">
                            <c:param name="action" value="editProduit"/>
                            <c:param name="id" value="${p.id}"/>
                        </c:url>
                        class="btn btn-primary">Modifier</a> |
                        <a href=
                        <c:url value="/Produit">
                            <c:param name="action" value="deleteProduit"/>
                            <c:param name="id" value="${p.id}"/>

```



```

                </c:url>
                class="btn btn-danger" onclick="return confirm('Voulez-vous vraiment
supprimer ce produit ?');">Supprimer</a>
            </td>
        </c:if>
    </tr>
</c:forEach>
</section>
</table>
</body>
</html>

```

Script 22 : home.jsp

c)- error.jsp :

```

<%@ page isErrorPage="true" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ page isELIgnored="false" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Error Page</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
integrity="sha384-FKYoEForCGlyvwx9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
crossorigin="anonymous"></script>
</head>
<body>

<h2>Error</h2>

<c:choose>
    <c:when test="${not empty exception}">
        <div class="card text-bg-danger container col-7">
            <p class="card-body display-6 text-center">${exception.message}</p>
        </div>
    </c:when>
    <c:otherwise>
        <div class="card text-bg-primary container col-7">
            <p class="card-body display-6 text-center">${message}</p>
        </div>
    </c:otherwise>
</c:choose>

<button class="btn btn-primary container col-5 text-center mt-2">
    <a href="<c:url value='/Produit'/">" class="text-decoration-none text-light">
        Retourner la page principale
    </a>
</button>

</body>
</html>

```

Script 23 : error.jsp



III. Conclusion :

Ce projet a permis de mettre en pratique une architecture web JEE structurée en plusieurs couches (Controller, Service, DAO et Vue), en adoptant des bonnes pratiques de conception telles que le Front Controller, les filtres et le pattern Unit of Work via Hibernate. L'utilisation de Hibernate a facilité la gestion de la persistance des données tout en assurant la cohérence des transactions grâce à une gestion centralisée des sessions. L'intégration de filtres pour la gestion des sessions et des accès a également renforcé la sécurité et la fiabilité de l'application.

Ainsi, ce travail a permis de mieux comprendre les mécanismes fondamentaux des applications web Java et l'importance d'une architecture bien structurée pour garantir la maintenabilité et l'évolutivité du système.

